sysinfo

Print the specifications of the machine you are on

User Manual · Version 1.0.0

`sysinfo` is a small command line program for Linux that reports the operating system, processor, memory, storage, graphics, network, sensors, user environment and the toolchain it was built with. It reads only `/proc`, `/sys` and a few system calls: no external programs, no root rights, no network access. The output is available as readable text, JSON or `key=value` lines.

Primary platform: Ubuntu 24.04 LTS and newer · C++20 · CMake $\geq$ 3.20

Contents

1 Quick start **3**

2 Installation **3**

2.1 Requirements . . . 3
2.2 Building . . . 3
2.3 Installing . . . 3
2.4 Building this manual . . . 4

3 Tests **4**

3.1 When the tests are available . . . 4
3.2 Running the tests . . . 4
3.3 The tests . . . 5
3.4 Limits of the tests . . . 6
3.5 Running the tests with sanitizers . . . 6
3.6 Adding a test . . . 6
3.7 Tests and continuous integration . . . 6

4 Command line reference **6**

4.1 Options . . . 7
4.2 Exit status . . . 8
4.3 Environment . . . 8
4.4 Examples . . . 8

5 Output formats **8**

5.1 Text . . . 8
5.2 JSON . . . 9
5.3 Key/value . . . 10

6 What is reported **10**

6.1 system — operating system and machine . . . 10
6.2 cpu — processor . . . 11
6.3 memory . . . 12
6.4 storage . . . 12
6.5 gpu — graphics . . . 12
6.6 network . . . 12
6.7 sensors — power and temperature . . . 13
6.8 environment . . . 13

6.9 build — how this program was built 13

7 Recipes **14**

7.1 Checking a machine before benchmarking 14

7.2 Recording the machine next to results 14

7.3 Comparing two machines 14

7.4 Using values in a script 14

7.5 Reporting a bug 14

8 Using the library **15**

9 How the information is obtained **16**

9.1 Limits to be aware of 16

10 Privacy **16**

11 Troubleshooting **17**

1 Quick start

```
sudo apt install build-essential cmake      # once
cmake -S . -B build                         # configure (Release by default)
cmake --build build -j                      # compile
./build/sysinfo                             # print everything
./build/sysinfo -s cpu,memory               # only some sections
./build/sysinfo -f json -o machine.json     # machine readable, to a file
```

That is all that is needed. The rest of this manual describes the options (Section 4), the tests (Section 3), the information that is reported (Section 6), how to embed the collector in your own program (Section 8) and what to do when something looks wrong (Section 11).

2 Installation

2.1 Requirements

- A C++20 compiler: GCC 11 or newer, or Clang 14 or newer with libstdc++. Ubuntu 24.04 ships GCC 13.
- CMake 3.20 or newer (Ubuntu 24.04 ships 3.28).
- For this manual only: a LaTeX installation with `latexmk` (`sudo apt install latexmk texlive-latex-recommended texlive-latex-extra`).

No third-party libraries are used.

2.2 Building

```
cmake -S . -B build
cmake --build build -j
ctest --test-dir build            # optional: 12 tests
```

If no build type is given, CMake configures a `Release` build. The build type and the compiler flags are recorded in the executable and reported in the `build` section.

CMake options:

Option	Default	Meaning
`SYSINFO_BUILD_CLI`	ON	build the `sysinfo` executable
`SYSINFO_BUILD_TESTS`	ON	register the tests for `ctest` (top-level builds only)
`SYSINFO_WARNINGS_AS_ERRORS`	OFF	compile with `-Werror` (`/WX` on MSVC)

Example with another compiler and a debug build:

```
cmake -S . -B build-clang -DCMAKE_CXX_COMPILER=clang++ -DCMAKE_BUILD_TYPE=Debug
```

2.3 Installing

```
sudo cmake --install build                          # to /usr/local/bin
cmake --install build --prefix "$HOME/.local"       # without root
```

2.4 Building this manual

```
cmake --build build --target manual                 # -> build/doc/sysinfo-manual.
    ↪ pdf
```

The target exists when `latexmk` is found at configure time. The example outputs in this manual are the files in `doc/examples/`, generated by the program itself.

3 Tests

The project ships 12 command line tests, registered with CTest in `CMakeLists.txt`. They run the compiled `sysinfo` executable and check its exit status and, for four of them, a pattern in its output. They are smoke and behaviour tests of the command line tool: they confirm that every mode runs on the current machine and that wrong input is rejected. They do not compare hardware values, so they pass on any computer.

3.1 When the tests are available

The tests are registered only when *all* of the following hold:

- `sysinfo` is the top-level CMake project (they are not registered when it is used from another project through `add_subdirectory`);
- `SYSINFO_BUILD_CLI` is `ON` (the tests need the executable);
- `SYSINFO_BUILD_TESTS` is `ON` (the default).

To switch them off: `cmake -S . -B build -DSYSINFO_BUILD_TESTS=OFF`.

3.2 Running the tests

Build first, then run CTest on the build directory:

```
cmake -S . -B build
cmake --build build -j
ctest --test-dir build                          # run all tests
```

A successful run ends with:

```
100% tests passed, 0 tests failed out of 12
```

The tests use the executable of the build directory; after changing the sources, rebuild before running them again. Useful variants:

Command	Effect
`ctest --test-dir build -N`	list the tests without running them
`ctest --test-dir build --output-on-failure`	print the program output of failed tests
`ctest --test-dir build -V`	verbose: show each command line and its output
`ctest --test-dir build -R json`	only tests whose name matches the regular expression
`ctest --test-dir build -E bad`	all tests except those matching
`ctest --test-dir build -j4`	run four tests in parallel
`ctest --test-dir build --rerun-failed`	repeat only the tests that failed last time

With a multi-configuration generator (Visual Studio, Xcode) add the configuration: `ctest --test-dir build -C Release`.

3.3 The tests

Exit 0 means the test passes when `sysinfo` exits with status 0. *Must fail* means the test passes when the exit status is not 0 (CTest property `WILL_FAIL`): these check that errors are detected. A *pattern* is a regular expression that must occur in the standard output (`PASS_REGULAR_EXPRESSION`); a test with a pattern passes only if the pattern matches, whatever the exit status.

Name	Command	Passes when / what it checks
`help`	`sysinfo --help`	exit 0. The usage text is printed.
`version`	`sysinfo --version`	output matches `^sysinfo <version>`, where `<version>` is the one given to `project()` in `CMakeLists.txt`.
`list`	`sysinfo --list`	exit 0. The section list is printed.
`text`	`sysinfo`	output matches `System.*Processor.*Memory`: all sections are collected and printed in text format, in display order.
`verbose`	`sysinfo --verbose`	exit 0. The long-tail collection (all flags, pseudo file systems, virtual interfaces) runs.
`json`	`sysinfo --format json`	output matches `"sys_os":` (key, colon, space); JSON is produced and contains the operating system key.
`kv`	`sysinfo -f kv -s core`	output matches `sys_cpu_logical_cores=[0-9]+`: key/value output with a numeric core count; selection by the `core` group.
`redacted`	`sysinfo -r -x build`	exit 0. Redaction combined with a section exclusion.
`bad_option`	`sysinfo --no-such-option`	must fail. An unknown option is rejected.
`bad_section`	`sysinfo -s nonsense`	must fail. An unknown section name is rejected.
`bad_format`	`sysinfo -f xml`	must fail. An unknown output format is rejected.

Name	Command	Passes when / what it checks
`unwritable_file`	`sysinfo -o /nonexistent-dir/out.txt`	must fail. An output file that cannot be created is reported as an error.

3.4 Limits of the tests

- The *must fail* tests accept any non-zero status, so they do not distinguish the documented usage error (2) from the I/O error (1). To verify the exact code run the command by hand and read `echo $?`; see Section 4.
- The pattern tests inspect only the standard output. A pattern is a regular expression in CMake syntax, so changing the output wording (for example a section title) requires updating the pattern in `CMakeLists.txt`.
- The tests do not verify that the reported values are correct for the hardware. Compare the output with `lscpu`, `free`, `lsblk` or `ip addr` for that.
- No test checks colour output, because the output is not a terminal under CTest.

3.5 Running the tests with sanitizers

A debug build with the compiler sanitizers is a good way to find memory errors and undefined behaviour in the file parsing code:

```
cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug -DSYSINFO_WARNINGS_AS_ERRORS=ON \
      -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-sanitize-recover=all"
cmake --build build-asan -j
ctest --test-dir build-asan --output-on-failure
```

3.6 Adding a test

Add a line to the test block of `CMakeLists.txt`, for example a test that the `gpu` section can be printed alone and mentions its title:

```
add_test(NAME gpu_only COMMAND sysinfo -s gpu)
set_tests_properties(gpu_only PROPERTIES PASS_REGULAR_EXPRESSION "Graphics")
```

Re-run CMake (it does so automatically on the next build) and check that the new test is listed with `ctest --test-dir build -N`.

3.7 Tests and continuous integration

CTest returns a non-zero exit status when any test fails, so the single command `ctest --test-dir build --output-on-failure` is enough in a CI job after the build step.

4 Command line reference

```
sysinfo [OPTIONS]
```

4.1 Options

Options may appear in any order. Short options can be combined (`-vr`) and may have their value attached (`-fjson`, `-scpu,memory`). Long options accept `--name value` and `--name=value`.

Option	Description
`-s, --section LIST`	Report only these sections. `LIST` is comma separated and the option may be repeated. Names: `system cpu memory storage gpu network sensors environment build`; plus `core` (system, cpu, memory, build) and `all`. Sections are always printed in the standard order, whatever the order on the command line.
`-x, --exclude LIST`	Leave these sections out; same syntax. Applied after `--section`.
`-l, --list`	Print the available sections (identifier, title, `(core)`) and exit.
`-v, --verbose`	Add the long tail: the full CPU flag list, swappiness and overcommit policy, pseudo file systems (`tmpfs`, ...), disks such as loop devices, virtual network interfaces and link-local IPv6 addresses.
`-f, --format FORMAT`	`text` (default), `json` or `kv`. See Section 5.
`-j, --json`	Shortcut for `--format json`.
`-o, --output FILE`	Write to `FILE` (created or truncated) instead of standard output. Colour is then off.
`-r, --redact`	Replace identifying values (hostname, user name, MAC and IP addresses) by `<redacted>`; see Section 10.
`--color[=WHEN]`	Colour the text format. `WHEN` is `auto` (default: only on a terminal, and not if `NO_COLOR` is set or `TERM=dumb`), `always` or `never`. Without a value: `always`.
`--no-color`	Same as `--color=never`.
`-h, --help`	Show the usage summary and exit.
`-V, --version`	Print `sysinfo 1.0.0` and exit.

The built-in help text:

```
Usage: sysinfo [OPTIONS]

Print the hardware and software specifications of this machine.

Selection:
  -s, --section LIST     only these sections (comma separated, repeatable);
                         names: system cpu memory storage gpu network sensors
                         environment build; also "core" and "all"
  -x, --exclude LIST     leave these sections out (same syntax)
  -l, --list             list the available sections and exit
  -v, --verbose          include the long tail: all CPU flags, pseudo
                         filesystems, virtual network interfaces, ...

Output:
  -f, --format FORMAT    text (default), json or kv (key=value lines)
  -j, --json             same as --format json
  -o, --output FILE      write to FILE instead of standard output
  -r, --redact           hide hostname, user name, MAC and IP addresses
      --color[=WHEN]     colour the text format: auto (default), always, never
      --no-color         same as --color=never (NO_COLOR is honoured as well)
```

```
Other:
  -h, --help              show this help and exit
  -V, --version           show the version and exit

Exit status: 0 success, 1 output could not be written, 2 usage error.
```

4.2 Exit status

Code	Meaning
0	success
1	the output file could not be opened, or writing failed (disk full, . . .)
2	usage error: unknown option or section, missing value, unexpected argument

A piece of information that cannot be determined never makes the program fail: it is shown as `unknown` or left out. Usage errors are reported on standard error.

4.3 Environment

`NO_COLOR`
: If set and non-empty, colour is not used in `auto` mode (see https://no-color.org).

`TERM`
: `dumb` disables automatic colour.

4.4 Examples

```
sysinfo -l                          # which sections exist?
sysinfo -s cpu                      # processor only
sysinfo -s core -x build            # system, cpu and memory
sysinfo -v -s cpu                   # processor, including all CPU flags
sysinfo -r > report.txt             # safe to paste into a bug report
sysinfo -j | jq '.sections.cpu.sys_cpu_model'
sysinfo -f kv -s memory | grep ram_total
```

5 Output formats

5.1 Text

Each section has a title; entries are listed as *label*, *value*. This is the format for reading. Example (hostname and other identifying values redacted, abbreviated virtual machine):

```
System
------
  Hostname                  <redacted>
  Operating system          Ubuntu 24.04.5 LTS
  Kernel                    Linux 6.18.44-fc-v139
  Architecture              x86_64
  Virtualization            yes (hypervisor flag set)
  Uptime                    0 h, 8 min
  Load average (1/5/15 min)  0.37  0.15  0.05
```

```
Processor
---------
  Model                          Intel(R) Xeon(R) Processor @ 2.10GHz
  Vendor                         GenuineIntel
  Family / model / stepping      6 / 207 / 2
  Microcode                      0x1
  Sockets                        1
  Physical cores                 2
  Logical cores (threads)        2
  SMT / Hyper-Threading          not supported
  NUMA nodes                     1
  Current frequency, cpu0 (MHz)  2100
  Instruction extensions         SSE2, SSE4.1, SSE4.2, AVX, AVX2, FMA, AVX-512F,
    ↪ AVX-512BW, AVX-512VL, AVX-512 VNNI, AMX, AES-NI, SHA, BMI2, POPCNT, VAES
  Caches (cpu0)                  L1d 48K, L1i 32K, L2 2048K, L3 266240K
  Cache line size                64 B

Memory
------
  Total RAM               7.8 GiB
  Available RAM           7.4 GiB
  Swap                    none
  Transparent huge pages  always [madvise] never
  Huge page size          2 MiB
  Page size               4 KiB

Storage
-------
  Disk vda                256.0 GiB, virtual disk
  Disk vdb                10 MiB, virtual disk
  Disk vdc                305 MiB, virtual disk
  Disk vdd                48 MiB, virtual disk
  Disk vde                692 KiB, virtual disk
  Disk vdf                6 MiB, virtual disk
  Mount /                 ext4 (/dev/vda), 252.0 GiB, 30% used
  Mount /opt/claude-code  ext4 (/dev/vdc), 304 MiB, 92% used
  Mount /opt/env-runner   ext4 (/dev/vdd), 47 MiB, 66% used

Build
-----
  Compiler          GCC 13.3.0
  Standard library  libstdc++ (GCC 13, 20240904)
  C++ standard      C++20 (202002L)
  Build type        Release
  Compiler flags    -O3 -DNDEBUG
  Pointer size      64 bit
  Byte order        little endian
  Assertions        disabled (NDEBUG)
  C library         glibc 2.39
  sysinfo version   1.0.0
```

5.2 JSON

One object with the program version, whether redaction was active, and a `sections` object that maps each section identifier to an object of *key* → *value* strings. All values are strings; text is valid UTF-8 (invalid bytes found in system files are replaced by U+FFFD).

```
{
  "sysinfo": "1.0.0",
  "redacted": true,
```

```
  "sections": {
    "memory": {
      "sys_ram_total": "7.8 GiB",
      "sys_ram_available": "7.4 GiB",
      "sys_swap": "none",
      "sys_thp": "always [madvise] never",
      "sys_hugepages": "2 MiB",
      "sys_page_size": "4 KiB"
    },
    "build": {
      "sys_compiler": "GCC 13.3.0",
      "sys_stdlib": "libstdc++ (GCC 13, 20240904)",
      "sys_cxx_std": "C++20 (202002L)",
      "sys_build_type": "Release",
      "sys_cxx_flags": "-O3 -DNDEBUG",
      "sys_pointer_bits": "64 bit",
      "sys_endianness": "little endian",
      "sys_assertions": "disabled (NDEBUG)",
      "sys_libc": "glibc 2.39",
      "sys_sysinfo_version": "1.0.0"
    }
  }
}
```


5.3 Key/value

One `key=value` per line, no quoting, in section order. Convenient for shell scripts and `grep`.


```
sys_hostname=<redacted>
sys_os=Ubuntu 24.04.5 LTS
sys_kernel=Linux 6.18.44-fc-v139
sys_arch=x86_64
sys_virtualized=yes (hypervisor flag set)
sys_uptime=0 h, 8 min
sys_loadavg=0.37  0.15  0.05
sys_cpu_model=Intel(R) Xeon(R) Processor @ 2.10GHz
sys_cpu_vendor=GenuineIntel
sys_cpu_signature=6 / 207 / 2
sys_cpu_microcode=0x1
sys_cpu_sockets=1
```


The keys are the same in all formats; they are stable and always start with `sys_`.

6 What is reported

All keys start with `sys_`; the tables omit the prefix. Entries marked † are optional: they are left out when the system does not provide them. Entries marked * are identifying and are hidden by `--redact`. Entries marked ᵛ appear only with `--verbose`.

6.1 **system** — operating system and machine

Key	Content
`sys_hostname`*	Host name.

Key	Content
sys_os	Distribution name and version from /etc/os-release, e.g. Ubuntu 24.04.5 LTS.
sys_kernel	Kernel name and release.
sys_arch	Machine architecture (x86_64, aarch64, ...).
sys_machine†	System vendor and product from DMI (or the device-tree model on ARM boards).
sys_board†	Mainboard vendor and name.
sys_firmware†	BIOS/UEFI vendor, version and date.
sys_virtualized	yes when the CPU reports a hypervisor or the DMI data names a virtual machine; otherwise no hypervisor flag.
sys_container†	Docker, Podman, Kubernetes, LXC, ..., when running inside one.
sys_wsl†	yes under Windows Subsystem for Linux.
sys_uptime	Time since boot.
sys_loadavg	Load average over 1, 5 and 15 minutes.

6.2 **cpu** — processor

Key	Content
sys_cpu_model	Marketing name of the processor.
sys_cpu_vendor†	Vendor string (GenuineIntel, AuthenticAMD, ...).
sys_cpu_signature†	x86 family / model / stepping.
sys_cpu_microcode†	Microcode revision.
sys_cpu_sockets	Number of processor packages.
sys_cpu_physical_cores	Number of physical cores (distinct hardware thread sibling groups).
sys_cpu_logical_cores	Number of online hardware threads.
sys_cpu_smt†	Simultaneous multithreading: on, off or not supported.
sys_cpu_numa_nodes†	Number of NUMA nodes.
sys_cpu_usable_cores†	Threads this process may run on, shown only when restricted (taskset, cpuset).
sys_cpu_cgroup_limit†	CPU quota of the container/cgroup, in CPUs.
sys_cpu_max_mhz† sys_cpu_min_mhz†	Frequency range allowed by the hardware, MHz (number only).
sys_cpu_cur_mhz†	Current frequency of cpu0, MHz (number only).
sys_cpu_governor†	Frequency governor of all CPUs; marked (mixed) if they differ.
sys_cpu_freq_driver†	Frequency scaling driver (intel_pstate, amd-pstate, ...).
sys_cpu_turbo†	Turbo/boost enabled or disabled.
sys_cpu_simd†	Notable instruction set extensions (SSE4.2, AVX2, AVX-512, AES-NI, NEON, SVE, ...).
sys_cpu_flags^{v}	The complete flag list from /proc/cpuinfo.
sys_cpu_caches†	Cache sizes of cpu0, e.g. L1d 48K, L1i 32K, L2 2048K, L3 266240K.

Key	Content
`sys_cpu_cache_line`†	L1 data cache line size.

6.3 memory

Key	Content
`sys_ram_total`	Installed RAM as seen by the kernel (GiB, one decimal).
`sys_ram_available`†	RAM available to new programs without swapping.
`sys_ram_cgroup_limit`†	Memory limit of the container/cgroup, when lower than the physical RAM.
`sys_swap`†	Swap size and use, or `none`.
`sys_thp`†	Transparent huge pages mode; the active one is in brackets.
`sys_hugepages`†	Huge page size and the number of reserved pages.
`sys_page_size`†	Virtual memory page size.
`sys_overcommit`ᵛ `sys_swappiness`ᵛ	Kernel tunables `vm.overcommit_memory` and `vm.swappiness`.

6.4 storage

Key	Content
`sys_disk_`*name*	One per block device: size, kind (`NVMe SSD`, `SSD`, `HDD`, `virtual disk`, `MMC/SD`) and model. Loop, RAM, optical and device-mapper devices only with `--verbose`.
`sys_fs_`*mount*	One per mounted file system: type, source device, size and percentage used. By default only file systems backed by storage (ext4, xfs, btrfs, zfs, vfat, ntfs, NFS, …) and the root overlay of a container; bind mounts of the same device are listed once. The key is derived from the mount point (`fs_root` for `/`).

6.5 gpu — graphics

Key	Content
`sys_gpu_`*n*	One per display controller on the PCI bus: vendor and model (looked up in the system's `pci.ids` database, or the NVIDIA driver's model name), kernel driver, and video memory where the driver exposes it (AMD). Without PCI graphics, DRM devices of system-on-chip boards are listed; otherwise `none detected`.
`sys_gpu_nvidia_driver`†	Version of the proprietary NVIDIA kernel module.

6.6 network

Key	Content
`sys_net_`*interface**	State, wireless marker, link speed, MTU, driver, MAC address and IPv4/IPv6 addresses with prefix length. With `--redact` the MAC and the addresses are removed, the rest is kept. The loopback device, Docker/KVM bridges and `veth` pairs are hidden unless `--verbose` is given.

6.7 **sensors** — power and temperature

Key	Content
`sys_battery_`*name*	Charge, status, technology, health (full charge relative to design capacity), cycle count, model.
`sys_ac_`*name*	Mains adapter connected or not.
`sys_temp_`*chip*	Temperature of each hardware monitor chip (`coretemp`, `k10temp`, `nvme`, ...); thermal zones are used when no chip is present.
`sys_sensors`	Shown instead when nothing is exposed (typical for virtual machines and containers).

6.8 environment

Key	Content
`sys_user`* `sys_is_root`	Effective user name and whether it is the superuser.

`sys_shell`
`sys_lang`
`sys_term`
	`$SHELL`, `$LANG`, `$TERM`.

`sys_session_type` `sys_desktop`	`$XDG_SESSION_TYPE` (x11/wayland) and `$XDG_CURRENT_DESKTOP`.
`sys_timezone`	From `/etc/timezone` or the `/etc/localtime` link.

Entries whose variable is unset are left out.

6.9 **build** — how this program was built

Key	Content
`sys_compiler` `sys_stdlib`	Compiler and C++ standard library with versions.
`sys_cxx_std`	C++ language level actually compiled, e.g. `C++20 (202002L)`.
`sys_build_type` `sys_cxx_flags`	CMake build type and the compiler flags in effect.

Key	Content
`sys_pointer_bits` `sys_endianness`	Word size and byte order of this binary.
`sys_assertions`	Whether `assert` is active (`NDEBUG`).
`sys_libc`†	C library version (glibc).
`sys_sysinfo_version`	Version of the collector.

7 Recipes

7.1 Checking a machine before benchmarking

Timing results are only comparable when the machine is in a known state. Look at:

```
sysinfo -s cpu,memory,sensors | grep -Ei 'governor|turbo|smt|frequency|limit|cur'
```

- `cpu_governor` should be `performance` and not `(mixed)`; `powersave` lets the clock drift.
- `cpu_turbo` and `cpu_smt` should be what you intend; decide once and keep it.
- `cpu_cgroup_limit`, `ram_cgroup_limit` and `cpu_usable_cores` reveal a container or `taskset` that restricts the program.
- `virtualized` `yes` means timings are subject to the host's scheduling.
- `build_type` must be `Release` or similar; `assertions` should be `disabled`.

7.2 Recording the machine next to results

```
sysinfo -j -o results/machine.json
sysinfo -f kv -s core -r > results/machine.kv
```

7.3 Comparing two machines

```
sysinfo -f kv -r -x environment,network > a.kv      # on machine A
sysinfo -f kv -r -x environment,network > b.kv      # on machine B
diff a.kv b.kv
```

7.4 Using values in a script

```
cores=$(sysinfo -f kv -s cpu | sed -n 's/^sys_cpu_physical_cores=//p')
make -j"$cores"
```

7.5 Reporting a bug

Attach the output of `sysinfo -v -r`; the hostname, user name, MAC and IP addresses are removed.

8 Using the library

The collector is a separate static library, `sysinfo::sysinfo`, that the command line tool is built on. Use it from another CMake project:

```
add_subdirectory(external/sysinfo)                  # or FetchContent
target_link_libraries(my_target PRIVATE sysinfo::sysinfo)
```

```
#include <sysinfo.h>

// structured: sections with titled entries
for (const sysinfo::Section& s : sysinfo::collectAll()) {
  std::cout << s.title << '\n';
  for (const sysinfo::Entry& e : s.entries)
    std::cout << "  " << e.label << ": " << e.display(/*redact=*/false) << '\n';
}

// flat: the "sys_*" pairs of the core sections
sysinfo::KeyValues kv = sysinfo::collect();
// e.g. with Google Benchmark:
for (const auto& [key, value] : kv) benchmark::AddCustomContext(key, value);
```

Function	Purpose
`version()`	Library version string.
`sections()`	Identifier, title and “core” flag of every section.
`collectSection(id, verbose)`	One section; an unknown identifier gives an empty section.
`collectAll(verbose)`	All sections in display order.
`collect()`	Flat list for the core sections (system, cpu, memory, build).

None of the functions throws; anything that fails is reported as `unknown`, left out, or (for a whole section) shown as a single `could not be collected` entry.

Build information. CMake generates `build_info.h` with the build type and compiler flags.

9 How the information is obtained

Section	Sources
system	`/etc/os-release`, `uname`, `/sys/devices/virtual/dmi/id/*`, `/proc/uptime`, `/proc/loadavg`, `/proc/1/cgroup`
cpu	`/proc/cpuinfo`, `/sys/devices/system/cpu/*` (topology, cpufreq, cache, smt, intel_pstate), `/sys/devices/system/node`, `sched_getaffinity`, `/sys/fs/cgroup/cpu.max`
memory	`/proc/meminfo`, `/sys/kernel/mm/transparent_hugepage`, `/sys/fs/cgroup/memory.max`, `sysconf`
storage	`/sys/block/*`, `/proc/self/mounts`, `statvfs`
gpu	`/sys/bus/pci/devices/*`, `pci.ids` (`/usr/share/hwdata`, `/usr/share/misc`), `/proc/driver/nvidia`, `/sys/class/drm`
network	`/sys/class/net/*`, `getifaddrs`
sensors	`/sys/class/power_supply`, `/sys/class/hwmon`, `/sys/class/thermal`
environment	environment variables, `getpwuid`, `/etc/timezone`

9.1 Limits to be aware of

- **No root, no external tools.** Information that needs administrator rights is not reported: memory module type and speed (`dmidecode`), serial numbers, SMART data, GPU load and clocks.
- **Virtual machines and containers** show what the guest sees. The CPU model may be generic, frequency and temperature files are often missing, and the cache sizes may be those of the host.
- **Caches** are read for cpu0 only. On hybrid processors (performance and efficiency cores) other cores can differ.
- **Current frequency** is a momentary reading of cpu0.
- **cgroup limits** are read for the program's own cgroup and the root; limits set on intermediate parent groups are not followed.
- **GPU names** depend on an up-to-date `pci.ids` (package `hwdata` or `pci.ids`); otherwise the numeric PCI identifier is shown.
- **Other platforms.** On macOS (via `sysctl`) and Windows (Win32 API) only the system, cpu, memory and build sections are filled, with fewer entries; the remaining sections say `not available on this platform`. These ports compile conditionally and have not been tested as thoroughly as Linux.

10 Privacy

`sysinfo` only reads local files and never sends anything anywhere. The output can nevertheless identify the machine or its user: the hostname, the user name, the MAC and IP addresses, the board and firmware version. `--redact` replaces `hostname` and `user` by `<redacted>` and removes the MAC and IP addresses from the network entries. It does *not* remove other details such as the exact machine model, the time zone or the kernel version; use `-x network,environment,system` to leave whole sections out when in doubt.

11 Troubleshooting

CMake: “Target requires the language dialect CXX20”
The compiler is too old. Use GCC 11+ or Clang 14+ (`-DCMAKE_CXX_COMPILER=g++-13`).

Many entries say `unknown`
The file under `/proc` or `/sys` does not exist or is not readable. This is normal in containers, virtual machines and restricted sandboxes. Compare with `sysinfo -v`.

No frequency, governor or turbo lines
The kernel exposes no `cpufreq` interface (virtual machine, or the driver is not loaded).

`cpu_physical_cores` equals `cpu_logical_cores`
The CPU has no SMT, SMT is disabled, or the hypervisor presents each thread as a core.

Garbage escape codes in a file or pipe
Colour is only enabled automatically on a terminal; add `--no-color` or set `NO_COLOR` when a wrapper forces it.

GPU shows only hexadecimal numbers
Install the PCI ID database: `sudo apt install hwdata` (or `pci.ids`).

Exit status 2
Read the message on standard error; `--help` lists the valid options and `--list` the sections.